Undecidability of Implication for Disjunctions of Dependence Atoms

24 October 2026

Notes

Miika Hannula (University of Tartu): This document is fully AI written. After iterative improvements on the lower bound, I suggested to investigate some kind of small model property for decidability, and if this fails to try identify some kind of obstacle. This led to the proof, after which I requested a couple minor modifications.

The research note was generated using ChatGPT (OpenAI, GPT-6 Astra, Ultra mode) under the ChatGPT for Academic Researchers complimentary account.

Abstract

We consider implication from a finite set of disjunctions of dependence atoms to one such disjunction, under team semantics and without a bound on the value domain. We prove that finite and unrestricted implication coincide and that their common decision problem is co-r.e.-complete. Hardness already holds for a fixed consequent with six atom occurrences, with at most five occurrences in each premise, and with only five variables exempt from explicit three-value bounds. The proof reduces two-counter machine halting to the existence of a finite counterteam. A Hall obstruction forces the complete incidence structure needed for the simulation; two matching channels then enforce control flow and exact counter updates. A three-marker construction compiles Boolean row conditions and conditional dependence requirements into the required pure dependence language. Consequently, every failure of implication has a finite counterteam, but no computable function of the input bounds the size of a smallest one.

1 The implication problem

We follow the standard team semantics of Väänänen [1, Chapter 3].

Definition 1.1 (Teams and dependence atoms)**.** Let V be a finite set of variables and D a nonempty value domain. An *assignment* is a function $s\colon V \to D$, and a *team* is a possibly empty set $T \subseteq D^V$ of assignments. For $X \subseteq V$ and $y \in V$, the *dependence atom* $=(X; y)$ states that the values of the variables in X determine the value of y. Precisely,

$$T \models =(X; y) \quad \iff \quad \forall s, t \in T \, [(\forall x \in X \; s(x) = t(x)) \Longrightarrow s(y) = t(y)].$$

For $X = \{x_1, \ldots, x_m\}$ we also write $=(x_1 \cdots x_m; y)$; the order of the determinant variables is immaterial. We write $=(y)$ for the constancy atom $=(\varnothing; y)$. It holds exactly when y has at most one value in T.

A *dependence disjunction* is a nonempty finite formula

$$\psi = \bigvee_{i=1}^{k} =(X_i; y_i), \qquad k \geq 1.$$

Its semantics is

$$T \models \psi \quad \iff \quad \exists T_1, \dots, T_k \subseteq T : \quad T = \bigcup_i T_i \ \text{ and } \ T_i \models {=}(X_i; y_i).$$

The parts may overlap and may be empty.[1] Repeated atom occurrences are retained: for example, $=(y) \vee =(y)$ allows two distinct y-values. There are no other atoms, constants, relation symbols, or first-order literals in the input language.

For a set Σ of formulas, $T \models \Sigma$ means that $T \models \psi$ for every $\psi \in \Sigma$.

Implication for dependence disjunctions

Input. A finite set Σ of dependence disjunctions and one dependence disjunction φ. Let V be the set of variables occurring in $\Sigma \cup \{\varphi\}$. All variables and atom occurrences are listed explicitly.

Question. Does $\Sigma \models \varphi$ hold? That is, is the following condition true?

$$\forall D \neq \varnothing \, \forall T \subseteq D^V : \qquad T \models \Sigma \implies T \models \varphi.$$

Finite-team variant. Ask the same question with T required to be finite; write $\Sigma \models_{\mathrm{fin}} \varphi$. In both variants the value domain D is unrestricted.

All equality tests compare values in the *same* variable; independently renaming the values of different variables does not change satisfaction.

Theorem 1.2 (Main result)**.** *Finite and unrestricted implication coincide. Their common decision problem is* co-r.e.*-complete under computable many-one reductions. Hardness holds already for the fixed consequent*

$$\varphi_* = {=}(w; z) \vee {=}(w; z) \vee {=}(U; w) \vee {=}(U; w) \vee {=}(V; w) \vee {=}(V; w).$$

Each premise has at most five atom occurrences. Moreover, every variable other than U, V, X, Y, w can be explicitly restricted to at most three values.

Here r.e. denotes the recursively enumerable sets and co-r.e. their complements. The hardness proof uses two-counter machine halting [4]. Sections 3–8 give the reduction, including the compilation into pure dependence formulas. We first establish finite counterteams and the upper bound.

Implication for this disjunction fragment was considered in Kontinen's thesis [2, Section 7.3]. The more restricted case involving disjunctions of two dependence atoms is raised as a further question by Fröhlich, Kolaitis, and Meier [3, Section 6]. The present reduction uses longer disjunctions and does not settle that two-occurrence restriction. Our argument is self-contained apart from propositional compactness, Hall's theorem, and the undecidability of two-counter machine halting.

2 Finite counterteams and enumeration

All formulas in the language are downward closed: if $S \subseteq T$ and $T \models \psi$, then $S \models \psi$. For an atom this is immediate; for a disjunction, intersect each part of a cover with S.

Lemma 2.1 (Finite character)**.** *For every formula ψ and team T,*

$$T \models \psi \quad \iff \quad \textit{every finite } S \subseteq T \textit{ satisfies } \psi.$$

[1] Requiring pairwise disjoint parts gives the same semantics: replace T_i by $T_i \setminus \bigcup_{j<i} T_j$. Each smaller team still satisfies its dependence atom. Thus strict and lax disjunction agree in this fragment.

Proof. Only the reverse implication needs proof. Write $\psi = \bigvee_{i=1}^{k} {=}(X_i; y_i)$. Introduce propositional variables $p_{s,i}$ for $s \in T$ and impose

$$\bigvee_{i=1}^{k} p_{s,i} \quad (s \in T), \qquad \neg p_{s,i} \vee \neg p_{t,i} \quad (s|_{X_i} = t|_{X_i},\ s(y_i) \neq t(y_i)).$$

Every finite subset of these clauses mentions finitely many rows, and a cover of that finite subteam satisfies the clauses. Propositional compactness supplies a model of all clauses. Its sets $T_i = \{s : p_{s,i} \text{ is true}\}$ form the required cover. □

Corollary 2.2. $\Sigma \models_{\mathrm{fin}} \varphi$ *if and only if* $\Sigma \models \varphi$.

Proof. If $T \models \Sigma$ but $T \not\models \varphi$, the lemma gives a finite $S \subseteq T$ with $S \not\models \varphi$. Downward closure preserves all premises on S. □

Lemma 2.3. *Nonimplication is recursively enumerable.*

Proof. Enumerate $r = 1, 2, \ldots$ and all teams on the input variables with values in $[r]$ and at most r rows. Every finite team appears up to independent renaming within each variable. For a given finite team, satisfaction of each formula is decidable by enumerating its row colourings. Stop upon finding a model of Σ that violates φ. Corollary 2.2 ensures termination on every nonimplication instance. □

We will also use projection invariance. If a formula uses only variables in A, it holds in T exactly when it holds in $\pi_A(T)$: covers project to the latter and lift back to the former. Injective renaming within each variable preserves all atoms and their disjunctions.

3 Compiling Boolean conditions into dependence disjunctions

Write $X \xrightarrow{\leq r} y$ for a disjunction of r copies of ${=}(X; y)$. It states that each X-fibre has at most r distinct y-values: each part permits one value, and conversely those values can be assigned to r parts separately in every fibre. This is an abbreviation, not an additional atom type.

3.1 Filtering complete three-marker fibres

Let A be a set of *semantic variables*, and let $z \notin A$ be a new marker variable. Under the premise $\varnothing \xrightarrow{\leq 3} z$, define

$$C_A(T) = \{a \in \pi_A(T) : |\{s(z) : s \in T,\ s|_A = a\}| = 3\}.$$

This is the *complete-fibre core*, a team on A. Auxiliary variables of T are ignored by this projection.

Lemma 3.1 (Filtering)**.** *If D uses only variables in A, then*

$$T \models {=}(A; z) \vee {=}(A; z) \vee D \quad \Longleftrightarrow \quad C_A(T) \models D.$$

Proof. Each ${=}(A; z)$-part covers at most one marker value per fibre. Thus at least one row in every complete fibre belongs to the parts for D. Their projections cover $C_A(T)$. Conversely, the first two parts can cover every incomplete fibre and two marker layers of every complete fibre. Lift a D-cover of the core to all remaining rows. □

3.2 Equality pairs encode three colours

For an ordinary graph-vertex variable c, impose

$$\varnothing \xrightarrow{\leq 2} c, \qquad =(Az; c), \qquad Ac \xrightarrow{\leq 2} z. \tag{1}$$

On a complete fibre, the three marker rows have exactly two c-values. They therefore select one of the three possible equal pairs. This pair is the vertex's colour. An edge between vertex variables c, d is encoded by

$$=(Acd; z). \tag{2}$$

On a complete fibre this holds exactly when their equal pairs differ: an identical pair violates the atom, whereas two different pairs have only singleton common equivalence classes.

Take anchor variables c_F, c_T, c_B, give each the conditions (1), impose their triangle using (2), and add $=(z; c_F)$, $=(z; c_T)$, and $=(z; c_B)$. If the core is nonempty, their three different pairs are independent of the semantic fibre. Rename the three markers so that

$$F = \{1, 2\}, \qquad T = \{1, 3\}, \qquad B = \{2, 3\}.$$

An edge to the anchor B restricts a vertex to the Boolean colours F, T. This encoding never compares values of different variables.

3.3 A consistent interpretation of semantic Boolean variables

Let $\mathcal{B} \subseteq A$ be the variables intended to be Boolean. For each $v \in \mathcal{B}$, impose $\varnothing \xrightarrow{\leq 2} v$ and introduce a switch variable w_v with

$$\varnothing \xrightarrow{\leq 3} w_v, \quad =(vz; w_v), \quad v \xrightarrow{\leq 2} w_v, \quad vw_v \xrightarrow{\leq 2} z, \tag{3}$$

$$=(c_F w_v; v), \qquad =(c_T w_v; v), \qquad =(Ac_B w_v; z). \tag{4}$$

Switches do not receive the ordinary vertex's global two-value bound. The variables in $A \setminus \mathcal{B}$ remain unrestricted.

Lemma 3.2 (Boolean decoding of a partial team)**.** *If the core is nonempty, each $v \in \mathcal{B}$ has a globally consistent injective interpretation*

$$\eta_v : \mathrm{rng}_{C_A(T)}(v) \longrightarrow \{0, 1\},$$

given by the switch colour F or T. Conversely, all switch conditions admit a canonical realization for any prescribed semantic Boolean values.

Proof. On a complete fibre, w_v has exactly two values: the bound $v \xrightarrow{\leq 2} w_v$ gives at most two, while $vw_v \xrightarrow{\leq 2} z$ excludes a constant value on all three markers. The edge to B makes its colour Boolean. The atom $=(vz; w_v)$ makes this marker triple depend only on the actual value of v.

Suppose two distinct v-values occur in the core. Their two-element sets of switch values intersect, because the switch has at most three values globally. If a shared value occurs at markers r, s in the two fibres, the atoms $=(c_F w_v; v)$ and $=(c_T w_v; v)$ require both $c_F(r) \neq c_F(s)$ and $c_T(r) \neq c_T(s)$. For the fixed anchor pairs this is possible only for $(r, s) = (2, 3)$ or $(3, 2)$. The shared value must therefore occur as a singleton in each triple, at opposite markers. The remaining equal pairs are the opposite Boolean colours F, T. This proves injectivity. Values occurring only in incomplete fibres need no interpretation.

For the canonical realization, use the anchor triples

$$c_F = (0, 0, 1), \quad c_T = (0, 1, 0), \quad c_B = (0, 1, 1),$$

and the switch triples

$$w_v = \begin{cases} (0,0,1), & v = 0, \\ (2,1,2), & v = 1. \end{cases}$$

Direct inspection verifies all conditions. These numbers are values in a constructed team, not constant symbols in the formulas. □

3.4 Arbitrary Boolean row conditions

A finite Boolean circuit can be converted into a polynomial-size 3CNF formula with auxiliary variables, preserving the extendability of each input assignment. For example, a gate $h = a \wedge b$ is expressed by $(\neg h \vee a)$, $(\neg h \vee b)$, and $(h \vee \neg a \vee \neg b)$. Negation and disjunction have analogous constant-size encodings; the output is required to be true. Repeating literals pads shorter clauses to length three.

Here is an explicit colouring graph for the resulting 3CNF. Each Boolean variable has vertices $v, \bar{v}$ with edges $v\bar{v}, vB, \bar{v}B$, forcing opposite Boolean colours. For a semantic bit, use its switch as the vertex v; all other vertices are ordinary auxiliary variables.

The condition $\mathrm{NAE}(p, q, r)$, meaning that three Boolean inputs are not all equal, is encoded by a new triangle t_1, t_2, t_3 and the edges t_1p, t_2q, t_3r. Equal inputs leave only two colours for the triangle. For unequal inputs, assign the opposite Boolean colour and B to the two vertices whose inputs agree, and the other Boolean colour to the third vertex. This is a proper colouring.

For a clause $\ell_1 \vee \ell_2 \vee \ell_3$, introduce a new Boolean vertex s with its complement and impose two such gadgets:

$$\mathrm{NAE}(\ell_1, \ell_2, s), \qquad \mathrm{NAE}(\bar{s}, \ell_3, F).$$

If all literals are false, the first condition forces $s = T$ and the second fails. If $\ell_3 = T$, the second is automatic and s can satisfy the first. If $\ell_3 = F$ but one of the other literals is true, choose $s = F$. Thus the graph extends a prescribed input colouring exactly when the clause holds.

Encode vertices and edges using (1)–(2). On each complete fibre, the resulting pure dependence premises force the desired Boolean row condition. Conversely, each satisfying input assignment extends to a local graph colouring. Give every ordinary vertex the canonical triple c_F, c_T, c_B for its colour. All auxiliary variables then have at most three values globally, regardless of the number of semantic fibres.

3.5 Conditional dependence and the compilation lemma

Lemma 3.3 (Conditional dependence)**.** *Let $b \in \mathcal{B}$ and let D use only semantic variables in A. The premise*

$$=(Ac_T w_b; z) \vee D$$

forces D on the subteam of the core where b is decoded as true. Conversely, the canonical three-marker expansion satisfies this premise whenever that selected semantic subteam satisfies D.

Proof. On a complete fibre with b true, c_T and w_b have the same pair. The first part cannot cover both rows of that pair, so at least one row belongs to the parts for D. Projection gives a D-cover of the selected core. Conversely, lift a D-cover to all true-b fibres. On a false-b fibre, c_T and w_b have different pairs, so (c_T, w_b) distinguishes all three markers. Assign all these fibres to the first part. The key A prevents interactions between different fibres. □

Lemma 3.4 (Compilation)**.** *Consider a finite semantic theory with variables A, Boolean variables $\mathcal{B} \subseteq A$, a Boolean row condition Γ, ordinary dependence disjunctions, and conditions of the form "the subteam selected by the Boolean condition P satisfies the dependence atom α". One can effectively construct a pure dependence disjunction theory $\widehat{\Sigma}$ such that:*

1. *the nonempty core of every $T \models \widehat{\Sigma}$ satisfies the semantic theory after Boolean decoding;*
2. *every finite semantic model has a finite canonical expansion satisfying $\widehat{\Sigma}$, with its prescribed Boolean guard extension as the core;*
3. *all new variables have explicit bounds of three values. In addition to the original disjunctions, at most three atom occurrences per premise are needed; a conditional requirement uses two.*

Proof. For each condition P, add a semantic Boolean variable b_P and include $b_P \leftrightarrow P$ in the row condition. From this point, A includes these guard variables. Use the colouring graph above for the combined row condition, switches for every semantic Boolean bit, and the marker bound $\varnothing \xrightarrow{\leq 3} z$. Retain the ordinary disjunctions and compile each conditional requirement by Lemma 3.3.

The first assertion follows from projection invariance, injectivity of Boolean decoding, and the constructions above. For the second, give each semantic row three markers, canonical anchors and switches, and a graph colouring extending its satisfying Boolean assignment. Lift ordinary covers to all three markers; use the preceding lemma for conditional covers. All auxiliary values are in $\{0,1\}$ or $\{0,1,2\}$, so explicit three-value bounds can be added. The construction has polynomial size in the input circuit description. □

4 A Hall obstruction forces complete ports

Write $k \cdot \alpha$ for a disjunction of k copies of α. First work on a finite semantic team R whose key is w: no two rows have the same w-value. View its rows as the edges of a bipartite multigraph. The variables U, V specify the endpoints, and w distinguishes parallel edges. Put

$$P_U = (4 \cdot {=}(U; w)) \vee {=}(w), \tag{5}$$
$$P_V = (4 \cdot {=}(V; w)) \vee {=}(w), \tag{6}$$
$$D = (2 \cdot {=}(U; w)) \vee (2 \cdot {=}(V; w)). \tag{7}$$

Lemma 4.1 (Capacity-two Hall obstruction)**.** *If $R \models P_U, P_V$ but $R \not\models D$, some subteam $S \subseteq R$ has exactly one U-vertex and one V-vertex of degree five, with every other occurring vertex of degree four. Moreover,*

$$|S| = 2\big(|\pi_U(S)| + |\pi_V(S)|\big) + 1.$$

Proof. A D-cover is equivalent to assigning each edge to one of two slots at its U-endpoint or one of two slots at its V-endpoint, without reusing a slot. Indeed, each of the four parts allows at most one edge at an endpoint of the corresponding sort. Hall's theorem [5] gives an edge set S with $|S| > 2(|\pi_U(S)| + |\pi_V(S)|)$.

Since w is a key, the constancy part of P_U contains at most one edge. Hence at most one U-vertex has degree five, and all others have degree at most four. The same holds on the V-side and is preserved in S. Writing $n = |\pi_U(S)| + |\pi_V(S)|$, we obtain

$$4n + 2 \;\leq\; 2|S| \;=\; \sum_v \deg_S(v) \;\leq\; 4n + 2.$$

Every inequality is equality, proving the assertion. □

5 The semantic counter-machine construction

Halting for two-counter machines is r.e.-complete: Turing machine halting many-one reduces to it [4, Lemma 3]. We use the following normal form. Both counters start at zero and are cleared before halting. Any prescribed initial counter values can be built by a finite initialization program,

and final values can be cleared by decrement/zero-test loops. The start state is fresh and has no incoming ordinary transition; the halt state is fresh and has no outgoing one. Duplicate states to record time parity. The start state has U-parity and the halt state V-parity; insert final no-op steps as necessary. Fresh intermediate states make each allowed ordered pair of states determine a unique operation and zero-test guard. The operations are increment, positive decrement, zero test, and no-op. A conditional zero/decrement instruction is represented by transitions to two distinct intermediate states. All transformations are effective and preserve halting.

The unbounded semantic variables are U, V, X, Y, w. The remaining semantic data

$$t, k_U, k_V, q_U, q_V, \delta$$

are finite-valued codes represented by Boolean bits. Here X, Y name configurations, U, V name their tokens, and q_U, q_V are control states. A token kind is one of

$$S,\ C_1,\ C_2,\ B,$$

meaning control token, a unit of the first or second counter, or a blank. Edge types are $a, b, c, d, e_1, e_2, e_3$. Their direction code is $\delta = 0$ for a, c, $\delta = 1$ for b, d, and $\delta = 2$ for e_i. Require valid codes throughout, with q_U, q_V belonging to the appropriate parity classes of states.

All conditions below are imposed on the semantic team. Determinant codes denote all their bits; output codes abbreviate one dependence atom per bit. Include the two degree premises (5)–(6) and the following conditions.

(1) The variable w determines every semantic variable, including all Boolean guard variables introduced by compilation. Impose

$$=(U;X),\quad =(U;k_U),\quad =(U;q_U),\quad =(V;Y),\quad =(V;k_V),\quad =(V;q_V),$$
$$=(X;q_U),\qquad =(Y;q_V),\qquad =(Ut;w),\qquad =(Vt;w).$$

Thus token data are consistent, and every token has at most one incident edge of each type.

(2) On rows with $k_U = S$, impose $=(X;U)$; on rows with $k_V = S$, impose $=(Y;V)$. Thus every configuration has at most one control token. On rows with $q_U =$ start impose $=(X)$, and on rows with $q_V =$ halt impose $=(Y)$. The start and halt configurations, if present, are globally unique. Only kinds S, B are allowed in these configurations.

(3) An e_i-edge is allowed only between the start configuration's U-control token and the halt configuration's V-control token. These two tokens forbid types b, d. Every other token permits only types a, b, c, d.

(4) Impose the dependence atoms

$$=(X\delta;Y),\qquad =(Y\delta;X).$$

For types a, c, the directed transition goes from (X, q_U) to (Y, q_V); for b, d, from (Y, q_V) to (X, q_U). The ordered state pair must be an allowed machine transition. The sole exception is a *reset* from halt to start on b, d-edges with both endpoint kinds B. No transition condition applies to the e_i-edges.

(5) The first channel, types a, b, requires $k_U = S \leftrightarrow k_V = S$. It does not otherwise restrict pairings between noncontrol token kinds.

(6) On the second channel, types c, d, the pair (L, R) of source and destination kinds follows the table below. Reset edges allow only (B, B). A zero test of counter j additionally forbids source kind C_j.

Operation	Allowed pairs (L, R)
No-op or zero test	$(S,S),(B,B),(C_1,C_1),(C_2,C_2)$
Increment j	$(S,C_j),(C_j,C_j),(C_{3-j},C_{3-j}),(B,B),(B,S)$
Decrement j	$(S,B),(C_j,C_j),(C_j,S),(C_{3-j},C_{3-j}),(B,B)$

The row conditions are Boolean circuits on the finite codes. The conditional requirements are covered by Lemma 3.4. There are no existential row requirements: all semantic conditions are preserved on subteams.

6 Extracting a halting computation from a counterteam

Suppose a finite semantic model R violates the core target D. Pass to the subteam S supplied by Lemma 4.1; all semantic conditions remain true. A degree-five token must use an e_i-edge, since there are only four ordinary edge types and each type occurs at most once at a token. The degree-five tokens are therefore exactly the globally unique start and halt control tokens. Both have all five allowed ports a, c, e_1, e_2, e_3. Every other token has exactly the four ports a, b, c, d.

Follow the first channel forward from the start control token, taking an a-edge at a U-token and a b-edge at a V-token. The kind remains S. Every internal control token has exactly one incoming and one outgoing edge in this directed channel. The start has no incoming edge and the halt has no outgoing one. The walk cannot enter a cycle: at the first repeated token, either the start acquires an incoming edge or another token acquires two different incoming edges of the same type. Finiteness therefore forces the walk to end at halt. Reset edges cannot occur on it, since both their endpoints have kind B.

The encountered configurations are distinct, since each has at most one control token and the walk does not repeat a token. Each of these configurations has exactly one control token. We claim that the second channel between consecutive configurations is a bijection on *their entire token sets*. For a transition from a U-configuration X_0 to a V-configuration Y_0, the control a-edge fixes the neighbour pair at direction 0. Every token of X_0 has a c-port, and $=(X\delta; Y)$ sends it to Y_0. Conversely, every token of Y_0 has a c-port, and $=(Y\delta; X)$ makes its predecessor belong to X_0. Port uniqueness gives a bijection. The argument for a transition in the other parity uses the d-ports and direction 1. The start and halt controls possess the required c-ports; every other token possesses all ordinary ports.

There is one S on either side of each bijection. For an increment of counter j, the source S becomes C_j and the unique target S comes from one blank. Thus the number of C_j-tokens increases by exactly one, while the other counter is unchanged. For a decrement, the source S becomes blank, and the target S requires one source C_j; hence the counter was positive and decreases by exactly one. The no-op preserves both counters. On a zero-test transition, all source tokens participate in the second channel, and its row guard excludes every C_j-token. The test is therefore genuinely zero. The control walk is a valid machine computation from zero counters to halt.

7 Constructing a counterteam from a halting computation

Conversely, take a halting computation of the normalized machine. Choose a positive integer N strictly larger than the maximum sum of the counter values along it. Represent each configuration by N tokens: one S, the required numbers of C_1, C_2, and the remaining blanks. Give every configuration a fresh X- or Y-identifier according to parity, and every token a fresh U- or V-identifier.

For each ordinary transition choose two independent bijections. The first maps S to S and pairs the remaining tokens arbitrarily. The second follows the operation table. Such a bijection exists: for an increment use the source control token for the additional counter unit and one

blank for the target control token; for a decrement use one counter unit for the target control token and the source control token for the additional blank. Match all remaining kinds identically. A zero test or no-op uses the identity on kinds. The choice of N supplies the blank needed by every increment. Assign edge types a, c or b, d according to the transition parity.

Both endpoint configurations have zero counters and hence $N - 1$ blanks. Join their blanks by two reset bijections of types b, d from halt to start. Add three edges of types e_1, e_2, e_3 between the start and halt control tokens. Give every edge a different w-value.

The atoms $=(X\delta; Y)$ and $=(Y\delta; X)$ hold because each configuration has one neighbour per direction, including reset and e_i directions. All other consistency and row conditions hold by construction. Every token has degree four except the two degree-five endpoint controls. To satisfy each degree premise, assign one e_i-edge to its constancy part; after removing that edge, all degrees on both sides are at most four. If the total number of tokens is n, there are $2n + 1$ edges. They cannot be assigned to the $2n$ available endpoint slots, so D fails.

8 The reduction in the required syntax

Apply Lemma 3.4 to the semantic theory of Section 5. Let A contain all semantic variables, including the Boolean guards, and call the resulting pure theory Σ_M. Its only potentially unbounded variables are U, V, X, Y, w. The marker z, all code bits, guards, switches, and graph variables have explicit bounds of at most three values. The key premises are $=(w; a)$ for $a \in A$.

Given a halting computation, Section 7 constructs a finite semantic model violating D. Its canonical expansion satisfies Σ_M, and Lemma 3.1 makes it violate $(2 \cdot =(A; z)) \vee D$. Conversely, a finite model of Σ_M violating this filtered target has a nonempty core violating D. By compilation and injective Boolean decoding the core satisfies all semantic conditions. Section 6 extracts a halting computation from it.

Since $w \in A$ and $=(w; a) \in \Sigma_M$ for all $a \in A$, agreement on w is exactly agreement on all of A, also on every subteam. Therefore $=(A; z)$ and $=(w; z)$ are interchangeable throughout a cover. The filtered target is equivalent, on all models of Σ_M, to the fixed formula φ_*. We have proved

$$M \text{ halts} \quad \Longleftrightarrow \quad \Sigma_M \not\models_{\text{fin}} \varphi_*.$$

This is an effective many-one reduction, so nonimplication is r.e.-hard. Lemma 2.3 gives r.e.-membership. Complementation proves co-r.e.-completeness, and Corollary 2.2 transfers the result to unrestricted teams.

The only five-occurrence premises are P_U and P_V. Conditional dependence requirements use two occurrences, and the remaining compiler premises use at most three. All the explicit three-value bounds are themselves three-occurrence disjunctions. This proves every restriction in Theorem 1.2.

Corollary 8.1 (No computable counterteam bound)**.** *There is no computable function $B(\Sigma, \varphi)$ such that every nonimplication instance has a counterteam of at most $B(\Sigma, \varphi)$ rows. This remains true for the fixed consequent and restricted premises of Theorem 1.2.*

Proof. A computable bound would decide implication by enumerating the finitely many equality types of teams up to that size and checking all premises and the consequent. This contradicts Theorem 1.2. □

Thus finite counterteams always suffice, but their sizes have no computable uniform bound in terms of the instance. The reason is not merely the existence of large minimal obstructions. Although the premises alone are downward closed, failure of the consequent supplies a Hall surplus. Combined with upper degree bounds, this surplus forces every required port to exist and hence enforces an entire computation of unbounded duration.

References

[1] Jouko Väänänen. *Dependence Logic: A New Approach to Independence Friendly Logic*. London Mathematical Society Student Texts 70, Cambridge University Press, 2007. doi:10.1017/CBO9780511611193.

[2] Jarmo A. Kontinen. *Coherence and Complexity in Fragments of Dependence Logic*. PhD thesis, Universiteit van Amsterdam, ILLC DS-2010-05, 2010. Thesis PDF.

[3] Nicolas Fröhlich, Phokion G. Kolaitis, and Arne Meier. *Disjunctions of Two Dependence Atoms*. In CSL 2026, LIPIcs 363, 10:1–10:21, 2026. doi:10.4230/LIPIcs.CSL.2026.10.

[4] Andrej Dudenhefner. *Constructive Many-One Reduction from the Halting Problem to Semi-unification*. In CSL 2022, LIPIcs 216, 18:1–18:19, 2022. doi:10.4230/LIPIcs.CSL.2022.18.

[5] Philip Hall. *On Representatives of Subsets*. Journal of the London Mathematical Society s1-10(1), 26–30, 1935. doi:10.1112/jlms/s1-10.37.26.